

Senior Backend Engineer Take-Home Assignment

Eclipse EDC - Policy-Aware Data Transfer Orchestration System

Assignment Overview

Estimated Time: 4-6 hours

Focus Areas: System Design, EDC Integration, Production-Ready Code

Submission Deadline: 7 days from receipt

Context

You're joining a team building a **Catena-X-compliant dataspace solution** using Eclipse EDC. Your first task is to design and implement a **Policy-Aware Data Transfer Orchestration System** that manages complex data transfers between multiple EDC connectors while respecting business policies, ensuring data sovereignty, and maintaining audit compliance.

Business Problem

In a Catena-X automotive supply chain network, manufacturers need to share sensitive data (supply chain metrics, quality reports, production forecasts) with their tier-1 and tier-2 suppliers. However:

1. **Different data types have different access policies**
 - Production data: Only accessible during business hours (8 AM - 6 PM)
 - Quality reports: Require a minimum quality certification level
 - Financial data: Restricted by geographical region (GDPR compliance)
 - Real-time metrics: Rate-limited (max 100 requests/hour per consumer)
2. **Transfers must be resilient and traceable**
 - Large files may fail mid-transfer
 - The system must support retry with exponential backoff
 - Complete audit trail required for compliance
 - Failed transfers should alert stakeholders

3. Multiple connectors in the ecosystem

- Provider may have multiple consumers
 - Each consumer has different policy requirements
 - System should optimise transfer orchestration
-

Your Task

Design and implement a **Transfer Orchestration Service** that acts as an intelligent layer on top of Eclipse EDC to:

1. **Evaluate complex policies** before initiating transfers
 2. **Orchestrate multi-step transfer workflows** with error handling
 3. **Track transfer lifecycle** with comprehensive audit logging
 4. **Provide transfer analytics** and monitoring capabilities
-

What You Need to Build

Part 1: System Design (40% weight)

Create an architecture document that includes:

1.1 Architecture Diagram

- Component diagram showing your orchestration service and its integration with EDC
- Sequence diagram for a complete transfer lifecycle
- Data flow diagram showing policy evaluation points

1.2 Design Decisions Document

Address these architectural concerns:

- **Policy Engine Design:** How will you evaluate complex, composable policies?
- **State Management:** How will you track transfer states across system restarts?
- **Failure Handling:** What's your retry strategy? Circuit breaker pattern?
- **Scalability:** How would this scale to 10,000 concurrent transfers?
- **Observability:** What metrics, logs, and traces will you expose?
- **Data Sovereignty:** How do you ensure GAIA-X compliance?

1.3 Technology Choices

Justify your choices for:

- State persistence (database/store)

- Message queue (if applicable)
- Caching strategy
- Monitoring/observability stack

1.4 API Design

Design REST/GraphQL APIs for:

- Initiating transfers with policy constraints
 - Querying transfer status
 - Retrieving audit logs
 - Getting transfer analytics
-

Part 2: Core Implementation (60% weight)

Implement the following components:

2.1 Policy Evaluation Engine ★ [Critical Component]

Create a flexible policy engine that can evaluate:

Java

```
// Example policy definitions you should support
TimeBasedPolicy: "Allow transfer only between 8 AM - 6 PM CET"
RateLimitPolicy: "Maximum 100 requests per hour per consumer"
GeographicPolicy: "Data must not leave EU region"
CertificationPolicy: "Consumer must have ISO 9001
certification"
UsagePolicy: "Data can only be used for quality analysis"
```

Requirements:

- Support policy composition (AND, OR, NOT operators)
- Policies should be stored in a database/file (your choice)
- Evaluate policies BEFORE initiating EDC contract negotiation
- Return clear policy violation reasons

Deliverable: `PolicyEvaluationService` with comprehensive tests

2.2 Transfer Orchestration Service ★ [Critical Component]

Implement the core orchestration logic:

Java

```
public interface TransferOrchestrator {  
    /**  
     * Initiates a data transfer with policy evaluation  
     * @return TransferRequest ID and initial status  
     */  
    TransferRequestResult initiateTransfer(TransferRequest  
request);  
  
    /**  
     * Gets current status of a transfer  
     */  
    TransferStatus getTransferStatus(String transferId);  
  
    /**  
     * Cancels an in-progress transfer  
     */  
    void cancelTransfer(String transferId);  
  
    /**  
     * Retrieves audit log for a transfer  
     */  
    List<AuditEvent> getTransferAuditLog(String transferId);  
}
```

Transfer Lifecycle States:

None

REQUESTED → POLICY_EVALUATION → APPROVED/DENIED →
CONTRACT_NEGOTIATION → NEGOTIATED → TRANSFER_IN_PROGRESS →
COMPLETED / FAILED / CANCELLED

Requirements:

- Integrate with EDC Management API for actual transfers
- Implement state machine for transfer lifecycle
- Store transfer history and state transitions
- Handle failures with exponential backoff retry
- Emit events for each state transition
- Transaction management for state updates

Deliverable: `TransferOrchestrationService` implementation

2.3 Audit & Compliance Module

Implement comprehensive audit logging:

```
Java
public interface AuditService {
    void logTransferRequest(TransferRequest request);
    void logPolicyEvaluation(String transferId,
        PolicyEvaluationResult result);
    void logStateTransition(String transferId, TransferState
        from, TransferState to);
    void logTransferCompletion(String transferId,
        TransferResult result);

    // Query capabilities for compliance reporting
    List<AuditEvent> getAuditTrail(String transferId);
    ComplianceReport generateComplianceReport(DateRange
        range);
}
```

Requirements:

- Immutable audit logs (append-only)
- Include timestamp, actor, action, metadata
- Support querying by transfer ID, date range, status
- Consider performance for high-volume scenarios

Deliverable: `AuditService` implementation

2.4 EDC Integration Layer

Create an abstraction over EDC Management API:

```
Java
public interface EdcConnectorClient {
    /**
     * Negotiates contract with consumer connector
     */
}
```

```

        */
        ContractNegotiationResult negotiateContract(ContractOffer
offer);

    /**
     * Initiates data transfer after successful negotiation
     */
    TransferProcessResult initiateTransfer(String agreementId,
TransferRequest request);

    /**
     * Monitors transfer process status
     */
    TransferProcessState getTransferState(String
transferProcessId);

    /**
     * Cancels ongoing transfer
     */
    void terminateTransfer(String transferProcessId);
}

```

Requirements:

- HTTP client for EDC Management API endpoints
- Error handling and retry logic
- Connection pooling
- Timeout configuration
- Response parsing and validation

Deliverable: `EdcConnectorClient` implementation with resilience

2.5 REST API Layer

Implement REST controllers for:

None		
POST	<code>/api/v1/transfers</code>	- Initiate transfer
GET	<code>/api/v1/transfers/{id}</code>	- Get transfer status

DELETE	/api/v1/transfers/{id}	- Cancel transfer
GET	/api/v1/transfers/{id}/audit	- Get audit log
GET	/api/v1/transfers (paginated)	- List transfers
GET	/api/v1/analytics/transfers	- Get transfer analytics
POST	/api/v1/policies/evaluate	- Test policy evaluation

Requirements:

- OpenAPI/Swagger documentation
- Input validation
- Proper HTTP status codes
- Error response format
- Pagination for list endpoints

Deliverable: REST Controllers with API documentation

Testing Requirements

Unit Tests (Mandatory)

- Policy evaluation logic (all policy types)
- State machine transitions
- Retry logic and backoff calculations
- Audit log generation

Integration Tests (Mandatory)

- End-to-end transfer flow (happy path)
- Policy denial scenarios
- Failure and retry scenarios
- EDC connector integration (can be mocked)

Test Coverage

- Aim for >75% code coverage
 - Focus on critical business logic
 - Include edge cases and error scenarios
-

Project Structure

Expected structure (adjust as needed):

```
None
transfer-orchestrator/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/company/orchestrator/
│   │   │       ├── api/                # REST controllers
│   │   │       ├── domain/            # Domain models
│   │   │       │   ├── model/
│   │   │       │   └── service/
│   │   │       └── infrastructure/    # EDC integration, DB,
etc.
│   │   │       │   ├── edc/
│   │   │       │   ├── persistence/
│   │   │       │   └── config/
│   │   │       ├── policy/            # Policy engine
│   │   │       └── audit/             # Audit service
│   │   └── resources/
│   │       ├── application.yml
│   │       └── db/migration/          # Database scripts
│   └── test/
│       └── java/
├── docs/
│   ├── architecture.md
│   ├── design-decisions.md
│   └── api-documentation.md
├── README.md
├── pom.xml / build.gradle
└── docker-compose.yml (optional)
```

Technical Requirements

Mandatory

- **Java 21+** (Java 25 preferred)
- **Spring Boot 3.5+** (4 preferred)

- **Latest version of Gradle**
- **Latest version of JUnit** for testing
- **Any database** (H2 for simplicity, PostgreSQL for production-ready approach, postgres preferred)

Recommended

- **SLF4J/Logback** for logging
- **OpenAPI** for API documentation

EDC Integration

- Integrate with actual **EDC connector**
 - Document your approach clearly
-

Deliverables Checklist

Code

- ☐ Compilable, runnable Java project
- ☐ All four critical components implemented
- ☐ Unit tests with >75% coverage
- ☐ Integration tests for main flows
- ☐ Clean commit history with meaningful messages

Documentation

- ☐ **README.md** with:
 - Setup instructions
 - How to run the application
 - How to run tests
 - API usage examples
 - Known limitations
- ☐ **architecture.md** with diagrams
- ☐ **design-decisions.md** explaining key choices
- ☐ **API documentation** (Swagger/OpenAPI)
- ☐ Inline code documentation (Javadoc for public APIs)

Architecture Artifacts

- ☐ Component diagram
- ☐ Sequence diagram (at least one key flow)
- ☐ Database schema (if applicable)
- ☐ Technology stack justification

Bonus Points (Optional)

These are NOT required but will be viewed very favorably:

Advanced Features (+15 points total)

- **Metrics & Monitoring** (+5): Integrate Prometheus metrics, custom dashboards
- **Event-Driven Architecture** (+5): Use message queue (Kafka/RabbitMQ) for state changes
- **Docker Compose Setup** (+3): Containerize application with dependencies
- **Performance Tests** (+2): JMeter/Gatling scripts for load testing

Production Readiness (+10 points total)

- **Health Checks** (+2): Spring Actuator with custom health indicators
- **Graceful Shutdown** (+2): Proper cleanup of in-progress transfers
- **Configuration Management** (+3): Externalized config, support for multiple environments
- **Database Migration** (+3): Flyway/Liquibase scripts

EDC Deep Integration (+10 points total)

- **Actual EDC Connector** (+5): Run real EDC connector and integrate
- **Custom EDC Extension** (+5): Build EDC extension that works with your orchestrator

Documentation & Presentation (+5 points total)

- **ADRs (Architecture Decision Records)** (+3): Document key decisions
- **Demo Video** (+2): 5-minute walkthrough of your solution

Evaluation Focus Areas

As a **senior engineer**, you'll be evaluated heavily on:

1. **System Design** (25%)
 - Architecture quality and scalability
 - Technology choices and trade-offs
 - Production readiness mindset
2. **Code Quality** (25%)
 - Clean code principles
 - SOLID principles

- Appropriate design patterns
 - Testability
 - 3. **EDC & Domain Knowledge (20%)**
 - Understanding of dataspace concepts
 - Policy enforcement approach
 - Data sovereignty considerations
 - 4. **Problem Solving (15%)**
 - Error handling and resilience
 - State management approach
 - Performance considerations
 - 5. **Communication (15%)**
 - Documentation quality
 - Design decision explanations
 - Code clarity and comments
-

Getting Started

Step 1: Review & Plan (30 minutes)

- Read this document thoroughly
- Sketch your initial architecture
- Identify unknowns and document assumptions

Step 2: Design (1-1.5 hours)

- Create architecture diagrams
- Write design decisions document
- Design API contracts

Step 3: Implement (2.5-3 hours)

- Set up project structure
- Implement core components
- Write tests as you go

Step 4: Polish (30-45 minutes)

- Write documentation
- Add examples
- Clean up code
- Final testing

Submission Instructions

What to Submit

1. **GitHub Repository** (preferred) or ZIP file containing:
 - Complete source code
 - Documentation in `/docs` folder
 - README.md in root
 - Any additional assets (diagrams, scripts)
2. **Include in email:**
 - Link to repository or attached ZIP
 - Any assumptions you made
 - Time spent on assignment
 - Anything you'd do differently with more time

Submission Format

Email Subject: `[Senior Backend Engineer] Take-Home Assignment - [Your Name]`

Repository Structure:

None

Please ensure:

- ✓ Code compiles and runs
- ✓ README.md has clear setup instructions
- ✓ Tests can be run with simple command
- ✓ No large binary files or build artifacts
- ✓ .gitignore is properly configured

FAQ

Q: Can I use AI assistants like ChatGPT/Copilot?

A: Yes, but be prepared to explain and defend all design decisions. We're evaluating YOUR thinking, not the AI's.

Q: What if I don't finish everything?

A: Focus on quality over quantity. Better to have 3 well-implemented components than 5 half-done ones. Document what you'd do with more time.

Q: Can I use libraries/frameworks not mentioned?

A: Absolutely! Just justify why you chose them.

Q: Should I implement a real database?

A: H2 in-memory is fine for this assignment. If you use PostgreSQL, docker-compose setup is appreciated.

Q: How detailed should the EDC integration be?

A: Mocking the EDC API is acceptable. Focus on your orchestration logic. Bonus points for real integration.

Q: Can I ask questions during the assignment?

A: Yes! Email [contact@company.com] with specific questions. We'll respond within 24 hours.

Q: What if I need more than 6 hours?

A: The time is a guideline. We value quality. However, if you're spending 15+ hours, you might be over-engineering. Document trade-offs.

What Happens Next

1. **Submission Review** (2-3 days)
 - Technical review by 2 senior engineers
 - Evaluation using our rubric
 2. **Technical Interview** (if selected)
 - 60-minute deep-dive into your solution
 - Discussion of design decisions
 - Potential enhancements and alternatives
 - System design discussion
 3. **Feedback**
 - All candidates receive feedback
 - Detailed evaluation of submission
-

Tips for Success

 Do:

- Start with architecture and design
- Write tests for critical logic
- Document assumptions clearly
- Show production-ready thinking

- Commit frequently with clear messages
- Focus on code quality over features

✗ Don't:

- Copy-paste code without understanding
 - Over-engineer simple problems
 - Ignore error handling
 - Skip documentation
 - Submit untested code
 - Leave TODOs without explanation
-

Contact

For questions or clarifications:

- **Email:** wahidul.azam@technovativesolutions.co.uk
 - **Response Time:** Within 24 hours on business days
-

Good luck! We're excited to see your solution!

This assignment is designed to simulate real-world challenges you'll face in our team. We're looking for engineers who can design robust systems, write clean code, and think critically about trade-offs.